

Ultralytics YOLO 超参数调优指南

简介

超参数调优不仅仅是一次性的设置，而是一个迭代过程，旨在优化机器学习模型的性能指标，如准确率、精确率和召回率。在 Ultralytics YOLO 的上下文中，这些超参数的范围可以从学习率到架构细节，例如层数或使用的激活函数类型。

How to Tune Hyperparameters for Better Model Performanc...



观看：如何调整超参数以获得更好的模型性能 🚀

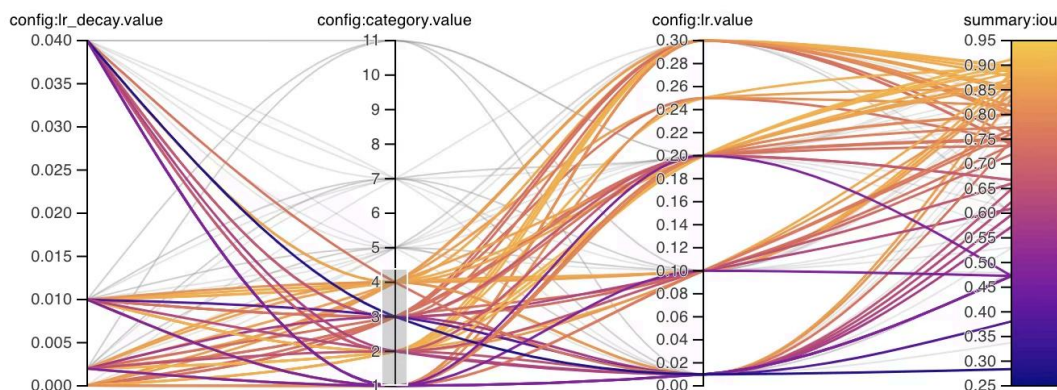
什么是超参数？

超参数是算法的高级结构设置。它们在训练阶段之前设置，并在训练期间保持不变。以下是 Ultralytics YOLO 中一些常用的调整超参数：

- **学习率** `lr0`：决定每次迭代中朝着最小值移动的步长 [损失函数](#)。
- **批量大小** `batch`：在前向传播中同时处理的图像数量。
- **迭代次数 (Epochs)** `epochs`：一个 epoch 是所有训练样本的一次完整的前向和后向传播。

Ask AI 

- **架构细节**：例如通道数、层数、激活函数类型等。



有关 YOLO11 中使用的增强超参数的完整列表，请参阅[配置页面](#)。

遗传进化和突变

Ultralytics YOLO 使用[遗传算法](#)来优化超参数。遗传算法的灵感来源于自然选择和遗传机制。

- **变异**：在 Ultralytics YOLO 的上下文中，变异通过对现有超参数应用小的随机变化，从而产生新的候选评估对象，从而有助于在局部搜索超参数空间。
- **交叉**：虽然交叉是一种流行的遗传算法技术，但 Ultralytics YOLO 目前不使用它进行超参数调优。重点主要放在变异上，以生成新的超参数集。

准备进行超参数调优

在开始调优过程之前，重要的是：

1. **确定指标**：确定用于评估模型性能的指标。这可以是 AP50、F1-score 或其他指标。
2. **设置调整预算**：确定您愿意分配多少计算资源。超参数调整可能需要大量的计算。

涉及的步骤

初始化超参数

首先从一组合理的初始超参数开始。这可以是 Ultralytics YOLO 设置的默认超参数，也可以是基于您的领域知识或之前的实验得出的。

突变超参数

使用 `_mutate` 方法，以基于现有集合生成一组新的超参数。该 `Tuner` 类 自动处理此过程。

训练模型

训练是使用经过变异的超参数集执行的。然后使用您选择的指标评估训练性能。

评估模型

使用 AP50、F1-score 或自定义指标等指标来评估模型的性能。[评估过程](#)有助于确定当前的超参数是否优于之前的超参数。

记录结果

记录性能指标和相应的超参数以供将来参考至关重要。Ultralytics YOLO 会自动以 CSV 格式保存这些结果。

重复

重复此过程，直到达到设定的迭代次数或性能指标令人满意为止。每次迭代都建立在从先前运行中获得的知识之上。

默认搜索空间描述

下表列出了 YOLO11 中用于超参数调整的默认搜索空间参数。每个参数都有一个由元组定义的特定取值范围 (min, max)。

参数	类型	值范围	描述
lr0	float	(1e-5, 1e-1)	训练开始时的初始学习率。较低的值提供更稳定的训练，但收敛速度较慢
lrf	float	(0.01, 1.0)	最终学习率因子，表示为lr0的比例。控制训练期间学习率的降低程度
momentum	float	(0.6, 0.98)	SGD 动量因子。较高的值有助于保持一致的梯度方向，并可以加速收敛
weight_decay	float	(0.0, 0.001)	L2 正则化因子，用于防止过拟合。值越大，正则化效果越强
warmup_epochs	float	(0.0, 5.0)	线性学习率预热的 epoch 数。有助于防止早期训练不稳定
warmup_momentum	float	(0.0, 0.95)	预热阶段的初始动量。逐渐增加到最终动量值

参数	类型	值范围	描述
box	float	(0.02, 0.2)	总损失函数中的边界框损失权重。平衡框回归与分类。
cls	float	(0.2, 4.0)	分类损失在总损失函数中的权重。值越高，越强调正确分类的预测。
hsv_h	float	(0.0, 0.1)	HSV 颜色空间中的随机色调增强范围。帮助模型泛化到各种颜色变化
hsv_s	float	(0.0, 0.9)	HSV 空间中的随机饱和度增强范围。模拟不同的光照条件
hsv_v	float	(0.0, 0.9)	随机值（亮度）增强范围。帮助模型处理不同的曝光水平
degrees	float	(0.0, 45.0)	最大旋转增强角度。帮助模型对对象方向保持不变。
translate	float	(0.0, 0.9)	最大平移增强，以图像尺寸的比例表示。提高对对象位置的鲁棒性。
scale	float	(0.0, 0.9)	随机缩放增强范围。帮助模型检测不同尺寸的物体
shear	float	(0.0, 10.0)	最大剪切增强角度。向训练图像添加类似透视的扭曲。
perspective	float	(0.0, 0.001)	随机透视增强范围。模拟不同的视角
flipud	float	(0.0, 1.0)	训练期间垂直图像翻转的概率。适用于高空/航拍图像。
fliplr	float	(0.0, 1.0)	水平图像翻转的概率。有助于模型对物体方向保持不变性。
mosaic	float	(0.0, 1.0)	使用 mosaic 增强的概率，它组合了 4 张图像。对小物体检测特别有用。
mixup	float	(0.0, 1.0)	使用 mixup 增强的概率，它混合了两张图像。可以提高模型的鲁棒性。
copy_paste	float	(0.0, 1.0)	使用复制粘贴增强的概率。有助于提高实例分割性能。

自定义搜索空间示例

以下是如何定义搜索空间并使用 `model.tune()` 方法来利用 `Tuner` 类，用于在 COCO8 上对 YOLO11n 进行 30 个 epoch 的超参数调整，使用 AdamW 优化器，并跳过绘图、检查点和验

证，仅在最后一个 epoch 上进行，以加快调整速度。

示例

Python

```
from ultralytics import YOLO

# Initialize the YOLO model
model = YOLO("yolo11n.pt")

# Define search space
search_space = {
    "lr0": (1e-5, 1e-1),
    "degrees": (0.0, 45.0),
}

# Tune hyperparameters on COCO8 for 30 epochs
model.tune(
    data="coco8.yaml",
    epochs=30,
    iterations=300,
    optimizer="AdamW",
    space=search_space,
    plots=False,
    save=False,
    val=False,
)
```

恢复中断的超参数调整会话

您可以通过传递以下参数来恢复中断的超参数调优会话 `resume=True`。您可以选择性地传递目录 `name` 在以下情况下使用 `runs/{task}` 恢复训练。否则，它将恢复上次中断的会话。您还需要提供所有之前的训练参数，包括 `data`, `epochs`, `iterations` 和 `space`。

 使用 `resume=True` 使用 `model.tune()`

```
from ultralytics import YOLO

# Define a YOLO model
model = YOLO("yolo11n.pt")

# Define search space
search_space = {
    "lr0": (1e-5, 1e-1),
    "degrees": (0.0, 45.0),
}

# Resume previous run
results = model.tune(data="coco8.yaml", epochs=50, iterations=300,
space=search_space, resume=True)

# Resume tuning run with name 'tune_exp'
results = model.tune(data="coco8.yaml", epochs=50, iterations=300,
space=search_space, name="tune_exp", resume=True)
```

结果

成功完成超参数调整过程后，您将获得多个文件和目录，其中封装了调整的结果。以下是每个文件和目录的描述：

文件结构

以下是结果的目录结构。像这样的训练目录 `train1/` 包含单独的调整迭代，即使用一组超参数训练的一个模型。这个 `tune/` 目录包含来自所有单独模型训练的调整结果：

```
runs/
├── detect/
│   ├── train1/
│   ├── train2/
│   ├── ...
│   └── tune/
│       ├── best_hyperparameters.yaml
│       ├── best_fitness.png
│       ├── tune_results.csv
│       ├── tune_scatter_plots.png
│       └── weights/
│           ├── last.pt
│           └── best.pt
```

文件描述

best_hyperparameters.yaml

此 YAML 文件包含在调整过程中找到的最佳性能超参数。您可以使用此文件来初始化具有这些优化设置的未来训练。

- **格式:** YAML
- **用法:** 超参数结果
- **示例:**

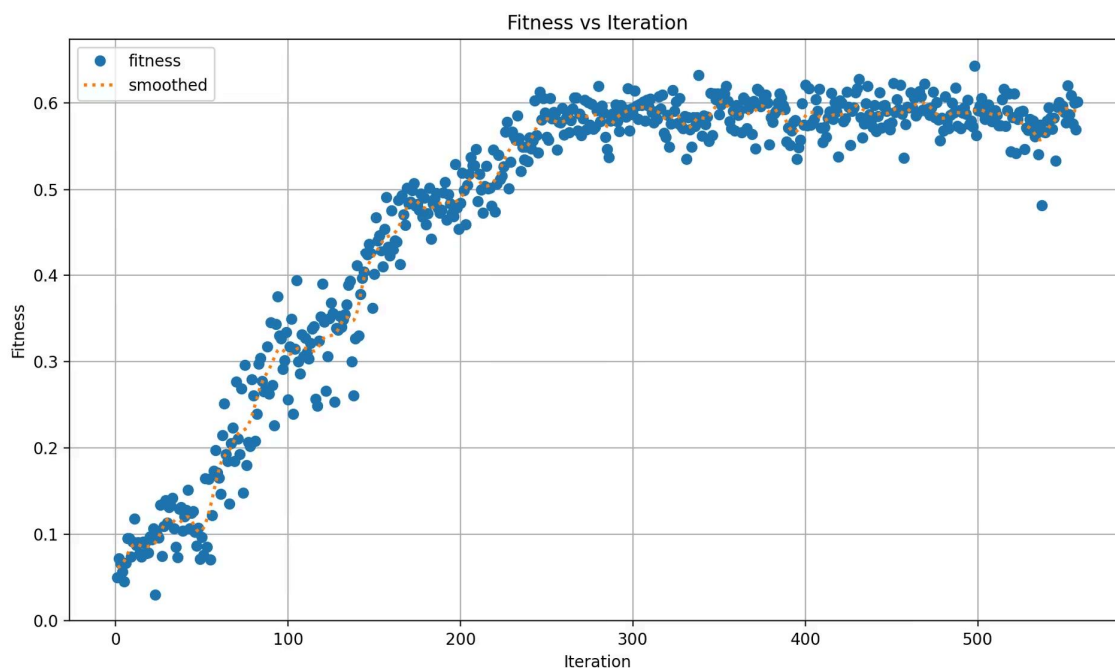
```
# 558/900 iterations complete ✅ (45536.81s)
# Results saved to /usr/src/ultralytics/runs/detect/tune
# Best fitness=0.64297 observed at iteration 498
# Best fitness metrics are {'metrics/precision(B)': 0.87247,
'metrics/recall(B)': 0.71387, 'metrics/mAP50(B)': 0.79106,
'metrics/mAP50-95(B)': 0.62651, 'val/box_loss': 2.79884,
'val/cls_loss': 2.72386, 'val/dfl_loss': 0.68503, 'fitness': 0.64297}
# Best fitness model is /usr/src/ultralytics/runs/detect/train498
# Best fitness hyperparameters are printed below.

lr0: 0.00269
lrf: 0.00288
momentum: 0.73375
weight_decay: 0.00015
warmup_epochs: 1.22935
warmup_momentum: 0.1525
box: 18.27875
cls: 1.32899
dfl: 0.56016
hsv_h: 0.01148
hsv_s: 0.53554
hsv_v: 0.13636
degrees: 0.0
```

best_fitness.png

这是一个图，显示了适应度（通常是像 AP50 这样的性能指标）与迭代次数的关系。它可以帮助您可视化遗传算法随时间的表现。

- **格式:** PNG
- **用法:** 性能可视化



tune_results.csv

一个 CSV 文件，包含调整期间每次迭代的详细结果。文件中的每一行代表一次迭代，它包括诸如适应度分数、[精确度](#)、[召回率](#)以及使用的超参数等指标。

- **格式:** CSV
- **用法:** 每次迭代的结果跟踪。
- **示例:**

```
fitness,lr0,lr1,lr2,momentum,weight_decay,warmup_epochs,warmup_momentum,box,cls,df1,hsv_h,hsv_s,hsv_v,degrees,translate,scale,shear,perspective,flipud,flipplr,mosaic,mixup,copy_paste

0.05021,0.01,0.01,0.937,0.0005,3.0,0.8,7.5,0.5,1.5,0.015,0.7,0.4,0.0,0.1,0.5,0.0,0.0,0.0,0.5,1.0,0.0,0.0

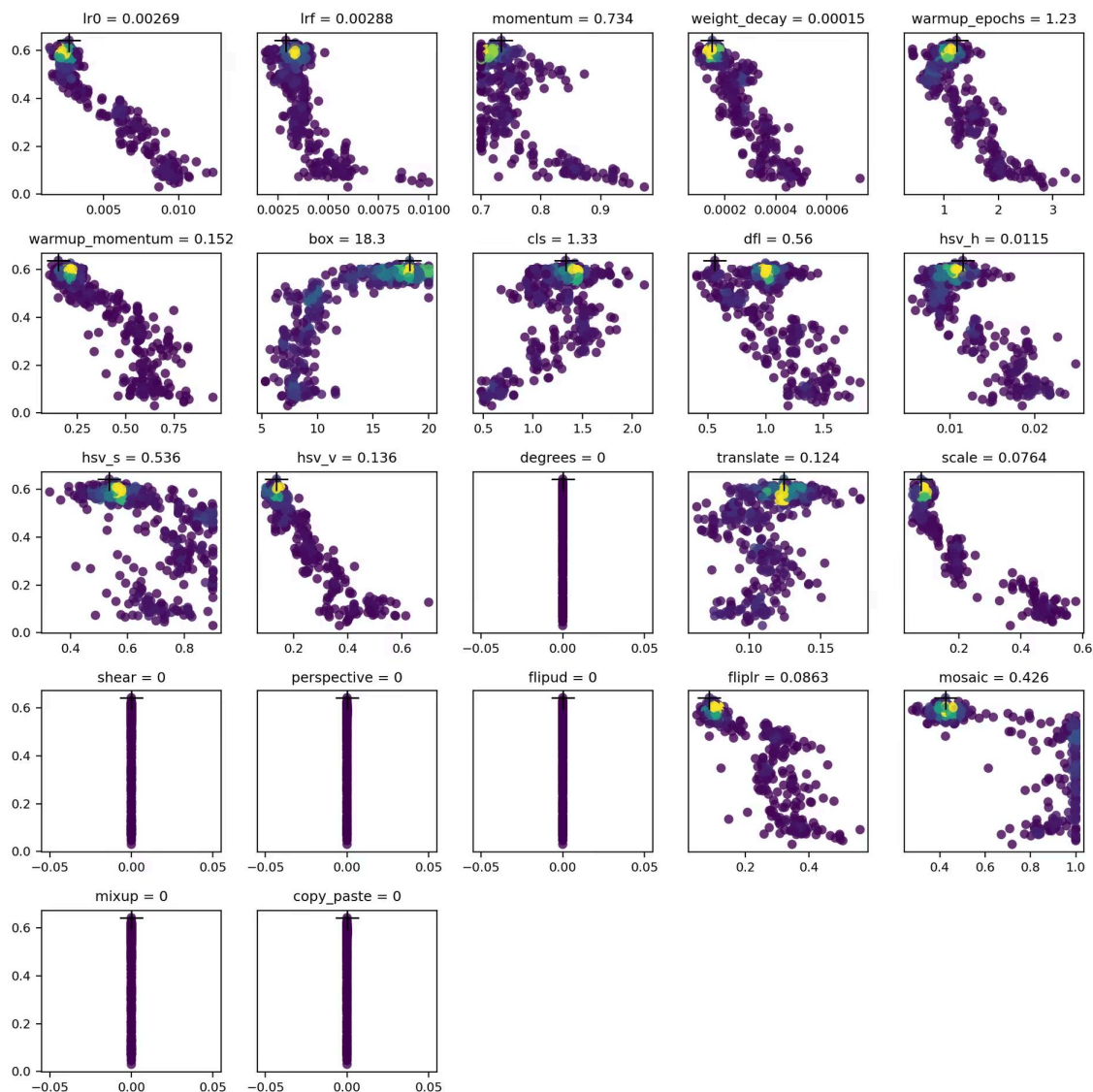
0.07217,0.01003,0.00967,0.93897,0.00049,2.79757,0.81075,7.5,0.50746,1.44826,0.01503,0.72948,0.40658,0.0,0.0987,0.4922,0.0,0.0,0.0,0.49729,1.0,0.0,0.0

0.06584,0.01003,0.00855,0.91009,0.00073,3.42176,0.95,8.64301,0.54594,1.72261,0.01503,0.59179,0.40658,0.0,0.0987,0.46955,0.0,0.0,0.0,0.49729,0.80187,0.0,0.0
```

tune_scatter_plots.png

此文件包含由此生成的散点图 `tune_results.csv`，帮助您可视化不同超参数和性能指标之间的关系。请注意，初始化为 0 的超参数将不会被调整，例如 `degrees` 和 `shear` 如下所示。

- 格式: PNG
- 用法: 探索性数据分析



weights/

此目录包含超参数调整过程中最后一次迭代和最佳迭代的已保存 [PyTorch](#) 模型。

- **last.pt** : last.pt 是来自训练的最后 epoch 的权重。
- **best.pt** : best.pt 是实现最佳适应度分数的迭代的权重。

使用这些结果，您可以为将来的模型训练和分析做出更明智的决策。请随时查阅这些文件，以了解您的模型表现如何以及如何进一步改进它。

结论

得益于 Ultralytics YOLO 中基于遗传算法并专注于突变的超参数调整方法，超参数调整过程得到了简化，但功能依然强大。遵循本指南中概述的步骤将有助于您系统地调整模型，从而获得更好的性能。

延伸阅读

1. [Wikipedia 中的超参数优化](#)
2. [YOLOv5 超参数演化指南](#)
3. [使用 Ray Tune 和 YOLO11 进行高效的超参数调整](#)

要获得更深入的了解，您可以浏览 [Tuner](#) 类 源代码和随附文档。如果您有任何问题、功能请求或需要进一步的帮助，请随时通过以下方式与我们联系 [GitHub](#) 或 [Discord](#)。

常见问题

如何在超参数调整期间优化 Ultralytics YOLO 的学习率？

要优化 Ultralytics YOLO 的学习率，请首先使用 `lr0` 参数设置初始学习率。常用值范围为 `0.001` 到 `0.01`。在超参数调整过程中，将改变此值以找到最佳设置。您可以利用 `model.tune()` 方法来自动执行此过程。例如：

示例

Python

```
from ultralytics import YOLO

# Initialize the YOLO model
model = YOLO("yolo11n.pt")

# Tune hyperparameters on COCO8 for 30 epochs
model.tune(data="coco8.yaml", epochs=30, iterations=300, optimizer="AdamW",
           plots=False, save=False, val=False)
```

有关更多详细信息，请查看[Ultralytics YOLO 配置页面](#)。

在 YOLO11 中，使用遗传算法进行超参数调优有什么好处？

Ultralytics YOLO11 中的遗传算法提供了一种强大的方法来探索超参数空间，从而实现高度优化的模型性能。主要优势包括：

- **高效搜索：**诸如突变之类的遗传算法可以快速探索大量的超参数。

- **避免局部最小值:** 通过引入随机性, 它们有助于避免局部最小值, 从而确保更好的全局优化。
- **性能指标:** 它们会根据性能指标进行调整, 例如 AP50 和 F1-score。

要了解遗传算法如何优化超参数, 请查看[超参数演化指南](#)。

Ultralytics YOLO 的超参数调优过程需要多长时间?

使用 Ultralytics YOLO 进行超参数调整所需的时间很大程度上取决于几个因素, 例如数据集的大小、模型架构的复杂性、迭代次数以及可用的计算资源。例如, 在 COCO8 这样的数据集上对 YOLO11n 进行 30 个 epoch 的调整可能需要几个小时到几天的时间, 具体取决于硬件。

为了有效地管理调整时间, 请预先定义一个明确的调整预算 ([内部章节链接](#))。这有助于平衡资源分配和优化目标。

在 YOLO 的超参数调优期间, 我应该使用哪些指标来评估模型性能?

在 YOLO 中进行超参数调整期间评估模型性能时, 可以使用以下几个关键指标:

- **AP50:** IoU 阈值为 0.50 时的平均精度。
- **F1-Score:** 精度和召回率的调和平均值。
- **精度和召回率:** 单独的指标, 指示模型在识别真阳性与假阳性和假阴性方面的[准确性](#)。

这些指标可帮助您了解模型性能的不同方面。有关全面概述, 请参阅 [Ultralytics YOLO 性能指标指南](#)。

我可以将 Ray Tune 用于 YOLO11 的高级超参数优化吗?

是的, Ultralytics YOLO11 与 [Ray Tune](#) 集成, 以实现高级超参数优化。Ray Tune 提供了复杂的搜索算法, 如贝叶斯优化和 Hyperband, 以及并行执行功能, 以加快调整过程。

要将 Ray Tune 与 YOLO11 结合使用, 只需设置 `use_ray=True` 中的参数 `model.tune()` 方法调用。更多详情和示例, 请查看 [Ray Tune 集成指南](#)。



创建于 1 年前



更新于 6 个月前



Tweet

分享